

Institute of Technology of Cambodia
Department of Applied Mathematics and Statistics



Introduction to Git

Advance Programming for Data Science

Date : 10, 2026

Presenter : Mr. VANN Visal

Table of Content

1. What is Git?
2. Key Concepts
3. Setting Up Git
4. Cloning an Existing Repository
5. Git Branch
6. Reverting Changes
7. Using .gitignore
8. Additional Resources



What is Git?

Overview:

What is Git?

Git is :

✓ Git is a distributed version control system.

✓ It helps track changes in source code during software development.

✓ Created by Linus Torvalds in 2005.

✓ Used for coordinating work among programmers, but it can be used to track changes in any set of files.

2. Key Concepts

▶ Repository (Repo): A database storing all the files and the history of changes.
▶ Commit: A snapshot of the repository at a specific point in time.
▶ Branch: A separate line of development.
▶ Merge: Combining changes from different branches.
▶ Clone: Copying a repository to your local machine.
▶ Push/Pull: Syncing changes between local and remote repositories.

3. Setting Up Git

▶ Install Git from https://git-scm.com/

▶ Configure Git with your name and email:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```



Cloning an Existing Repository

4. Creating a New Repository

▶ Navigate to your project directory:

```
cd /path/to/your/project
```

▶ Initialize a new Git repository:

```
git init
```

5. Cloning an Existing Repository

▶ Clone a repository from a URL:

```
git clone https://github.com/user/repo.git
```

This creates a directory named repo containing all the files and history.

6. Checking the Status

▶ Check the status of your files:

```
git status
```

This shows which files are staged, unstaged, or untracked.

7. Adding Files to Staging Area

- Add a specific file to the staging area:

```
git add filename
git add filename_1 filename_2
```

- Add all files to the staging area:

```
git add .
```

- Add all files that the same type to the staging area:

```
git add *.extension
```

8. Renaming Files & Moving Files

- Rename a file:

```
git mv old_filename new_filename
```

- Move a file to a different directory:

```
git mv filename path/to/new_directory/
```

9. Deleting Files

- Delete a file from the repository:

```
git rm filename
```

- Forcefully remove a file from the working directory and the index:

```
git rm -f filename
```

- Forcefully remove a directory and its contents:

```
git rm -rf directoryname
```

Use these commands with caution, as they will delete the specified files or directories from your working directory and the staging area.

10. Committing Changes

▶ Commit staged changes with a message:

```
git commit -m "Your commit message"
```

This creates a new commit in the repository.

▶ To change commit's message:

```
git commit --amend
```

11. Viewing Commit History

▶ View the commit history:

```
git log
```

This shows a list of commits with their messages, authors, and dates.

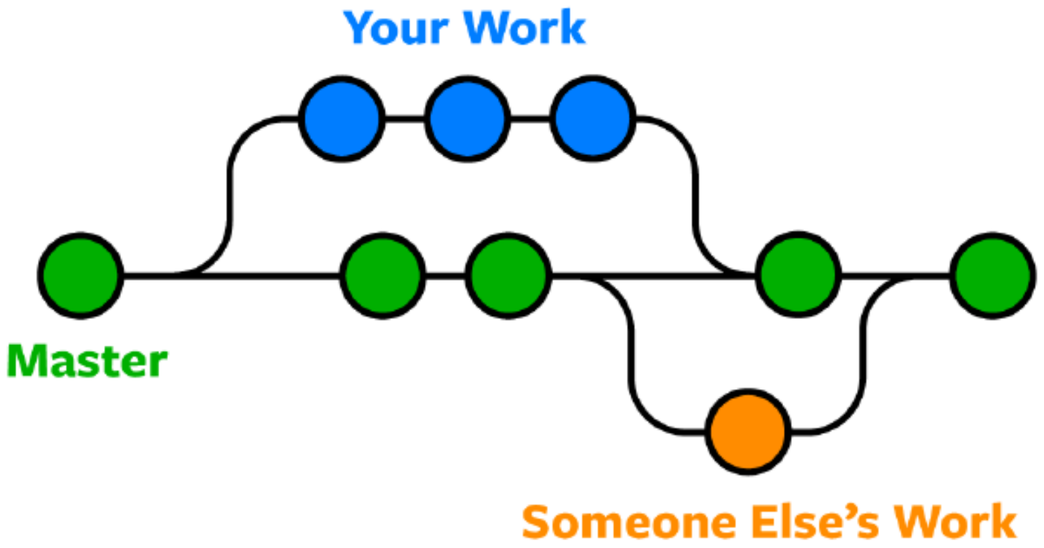
▶ We also can use this to view as short list:

```
git log --oneline
```



Git Branch

12. Git Branch



13. Branching

- Create a new branch:

```
git checkout -b new-branch
```

- Switch to an existing branch:

```
git checkout existing-branch
```

14. Switching Branches

▶ Switch to an existing branch:

```
git switch branch-name
```

▶ Create and switch to a new branch:

```
git switch -c new-branch-name
```

- The git switch command is an alternative to git checkout for changing branches.
- Use this command to easily switch between branches or create and switch to a new branch.

15. Swapping Branches to Make Another Branch the Main Branch

▶ Rename the current main branch (e.g., main) to another name:

```
git branch -m main old-main
```

▶ Rename the desired branch (e.g., feature-branch) to main:

```
git branch -m feature-branch main
```

▶ Update the remote repository:

```
git push origin -u main
```

▶ Delete the old main branch from the remote repository:

```
git push origin --delete old-main
```

- If the main branch is protected, change the default branch in the repository settings first.
- These commands rename the branches locally and update the remote repository.

16. Merging Branches

- Merge a branch into the current branch:

```
git merge branch-to-merge
```

Resolve any conflicts that arise.

17. Deleting Branches

- Delete a branch locally:

```
git branch -d branch-to-delete
```

- Delete a branch on the remote repository:

```
git push origin --delete branch-to-delete
```

18. Syncing with Remote Repository

- Push changes to the remote repository:

```
git push origin branch-name
```

- Pull changes from the remote repository:

```
git pull origin branch-name
```

Resolving Conflicts

Resolving Conflicts

- Conflicts occur when changes in different branches are incompatible.
- Git marks the conflicts in the affected files.
- Edit the files to resolve conflicts, then add and commit the changes.

19. Reverting Changes

▶ Revert a file to a previous commit:

```
git checkout commit-id -- filename
```

▶ Revert the entire repository to a previous commit:

```
git reset --hard commit-id
```

▶ Revert a commit by creating a new commit that undoes the changes:

```
git revert commit-id
```

- This creates a new commit with the inverse of the specified commit.
- Useful for undoing changes without altering the commit history.

20. Stashing Changes

- Stash changes without committing them:

```
git stash
git stash push -m "Your stash message"
```

- Apply stashed changes:

```
git stash apply
git stash apply index-stash
```

- View stashed changes:

```
git stash list
```

20. Stashing Changes

- Drop stashed changes:

```
git stash drop
git stash drop index-stash
```

- Finalize stashed changes:

```
git stash pop index-stash
```

- Clear the stashed changed list:

```
git stash clear
```



.gitignore

21. Using .gitignore

- The .gitignore file tells Git which files (or patterns) it should ignore.
- Create a .gitignore file in your repository's root directory.
- Common uses:
  - Ignoring files generated by the operating system.
  - Ignoring log files and temporary files.
  - Ignoring build artifacts.

Example for a Python project

curl -L -o .gitignore

https://www.toptal.com/developers/gitignore/api/python

22. Getting Help with Git Commands

- Display general help:

```
git --help
```

- Get help for a specific Git command:

```
git <command> --help
```

- Examples:
  - Get help with the clone command:

```
git clone --help
```

  - Get help with the status command:

```
git status --help
```

- These commands will display the manual page for the specified command, providing detailed information and usage examples.

23. Common Problems and Solutions

▶ Merge Conflicts: Occur when two branches have changes in the same place. Resolve by editing the conflicting files and committing the changes.

▶ Detached HEAD State: Happens when you check out a commit instead of a branch. Fix by checking out a branch:

```
git checkout branch-name
```

▶ Forgotten Files in a Commit: Add the files and amend the commit:

```
git add forgotten-file
git commit --amend
```

24. Tips for Effective Git Usage

- Write meaningful commit messages.
- Commit often with small, incremental changes.
- Use branches to isolate features and fixes.
- Regularly pull from the remote repository to stay updated.
- Rebase your changes before merging to keep a clean history.

24. Tips for Effective Git Usage

- Communication with your teammate about changing.
- Define tasks which edit the different files.

```
git status
git add filename
git commit -m "Your commit's message"
git pull
<In case, we commit the same file>
<We can merge in the editor such as VS Code>
git add filename
git commit -m "Your commit's message"
git push
```

Practices

Working as a Team with Git (Part 1)

▶ Example Team Workflow:

1. Clone the repository:

```
git clone https://github.com/user/repo.git
```

2. Create a new branch for your feature:

```
git switch -c feature-branch
```

3. Make changes and commit them:

```
# Edit files
git add .
git commit -m "Add new feature"
```

4. Push your branch to the remote repository:

```
git push origin feature-branch
```

Working as a Team with Git (Part 2)

▶ Example Team Workflow:

5. **Create a pull request:**
   - Go to the repository on GitHub (or another Git platform).
   - Click ”Compare pull request”.
   - Review your changes and create the pull request.
6. **Review and merge the pull request:**
   - Team members review the pull request.
   - After approval, merge the pull request into the main branch.
7. **Update your local main branch:**

   ```
   git switch main
   git pull origin main
   ```

   - This workflow ensures that everyone works on their own branches, reducing conflicts and facilitating code reviews.
   - Regularly pulling updates keeps your local repository in sync with the remote repository.

Additional Resources

- Git Guide Book
- Learning Branching
- Atlassian
- W3School

THANK YOU!



AI IS

Department of Applied Mathematics and Statistics